

Assignment 1

Must use: the concepts listed for that assignment (inheritance, interface, encapsulation, polymorphism, exception handling, etc.).

Data layer: In-memory lists/maps are fine.

CLI or simple menu is enough.

1) Hospital Management System

Concepts: Inheritance, Aggregation, Encapsulation, Exception Handling

Learning goals

- Model a staff/patient hierarchy, protect sensitive fields, enforce scheduling rules.

Scope & features

- Register Doctor, Patient, Nurse (derived from Person).
- Book appointments (avoid overlaps per doctor).
- Prescribe medicines (doctor → patient).
- Generate bills from appointments/meds.
- Sensitive data (medical history) via encapsulation.

Classes:

- Person (base) → Doctor, Patient, Nurse (derived)
- Appointment, Medicine, Bill

Features:

- Book appointments, issue bills, and prescribe medicines.
- Use exception handling for invalid inputs (e.g., appointment overlaps).
- Encapsulate sensitive data like patient records.

2) Online Payment Gateway

Concepts: Interface, Polymorphism, Exception Handling

Goals

- Strategy pattern via PaymentMethod interface. Validate inputs; surface custom errors.

Scope

- Methods: authorize(amount), capture(amount), refund(amount).
- Implementations: CreditCard, PayPal, UPI, BankTransfer.
- Custom exceptions: InsufficientBalanceException, InvalidCardException, AuthorizationFailedException.

Classes:

- Interface: PaymentMethod
- Classes: CreditCard, PayPal, UPI, BankTransfer

Features:

- Each payment class implements its own makePayment() method.
- Throw custom exceptions (e.g., InsufficientBalanceException, InvalidCardException).
- Demonstrate polymorphism by processing payments dynamically.

3) Smart Home Automation System

Concepts: Abstraction, Interface, Polymorphism

Goals

- Interface-based design; central orchestrator (Facade) as SmartHub.

Scope

- Device interface with turnOn(), turnOff(), setTimer(Duration).
- Implementations: Light, Fan, Thermostat, DoorLock.
- SmartHub registers devices, schedules timers.

Classes:

- Interface: Device → Light, Fan, Thermostat, DoorLock

Features:

- Each device implements turnOn(), turnOff(), and setTimer() differently.
- Add abstraction by creating a central controller class SmartHub.
- Demonstrate interface-based programming and device polymorphism.

4) University Course Management

Concepts: Inheritance, Polymorphism, Encapsulation

Goals

- Course types override submission rules; grades encapsulated.

Scope

- Entities: Course, OnlineCourse, OfflineCourse, Student, Professor, Assignment, Grade.
- Encapsulation: only professor can set grade; students can only view their own.

Classes:

- Course, Student, Professor, Assignment, Grade

Features:

- Allow adding and grading assignments.
- Subclasses for different course types (OnlineCourse, OfflineCourse) that override methods like submitAssignment().
- Secure access to grades using encapsulation.

5) E-commerce Shopping Cart

Concepts: Inheritance, Polymorphism, Exception Handling

Goals

- Product subclasses with pricing rules; stock validation; order flow.

Scope

- Product → Electronics, Clothing, Grocery (override priceAfterDiscount()).
- ShoppingCart add/remove, Order generate invoice.
- Exceptions: OutOfStockException, InvalidProductException.

Classes:

- Product → Electronics, Clothing, Grocery
- ShoppingCart, User, Order

Features:

- Add/remove items and apply discounts.
- Use polymorphism for different product pricing rules.
- Use exceptions for stock or invalid product ID handling.

6) Car Rental System

Concepts: Inheritance, Interface, Polymorphism

Goals

- Uniform Rentable interface; vehicle types compute rent differently.

Scope

- Vehicle base → Car, Bike, Truck; Rentable with rent(days) → RentalAgreement.
- Extras: insurance, mileage tiers.

Classes:

- Vehicle (base) → Car, Bike, Truck
- Interface: Rentable

Features:

- Each vehicle type has a different rent calculation method.
- Implement interface for rental processing.
- Use polymorphism when renting any vehicle type.

7) Banking Application

Concepts: Encapsulation, Inheritance, Exception Handling

Goals

- Secure balance through getters/setters; different interest/fees by account type.

Scope

- Account → SavingsAccount, CurrentAccount.
- deposit, withdraw, transfer.

- Exceptions: InsufficientFundsException, NegativeAmountException.

Classes:

- Account (base) → SavingsAccount, CurrentAccount
- Transaction, Customer

Features:

- Use getters/setters to secure balance details.
- Handle exceptions like InsufficientFundsException.
- Demonstrate inheritance for account types with unique interest calculations.

8) Simple 2D Game (OO)

Concepts: Polymorphism, Inheritance, Encapsulation

Goals

- Characters share base behaviors; attacks differ polymorphically.

Scope

- Character base → Player, Enemy, Boss.
- Weapon, PowerUp modify damage/time.
- Encapsulate health, level with controlled mutation.

Classes:

- Character (base) → Player, Enemy, Boss
- Weapon, PowerUp

Features:

- Use polymorphism for attack styles.
- Encapsulate player stats (health, level).
- Inherit movement and combat behavior.